

《微机原理实验》实验报告

实验名称：并行端口 & LCD 显示 指导教师：肖山林

姓名：徐睿琳 学号：23342107 专业/班级：微电子/三班 分组序号：A412-A05

实验日期：2025.12.17 实验地点：教学楼 A412 是否调课/补课：否 成绩：

目录

1 实验内容与设计	2
1.1 8255A 控制 LED 显示	2
1.2 LCD 显示字符	3
1.3 8255A 控制 7 位数码管显示	7
2 实验结果	9
2.1 8255A 控制 LED 显示	9
2.2 LCD 显示字符	9
2.3 8255A 控制 7 位数码管显示	10
3 思考与总结	10
3.1 实验总结	10
3.2 鲁棒性提升方案：LCD 忙标志检测	11
3.2.1 问题分析	11
3.2.2 改进代码实现	11
附录 A 学号-姓名字符表	13

1 实验内容与设计

1.1 8255A 控制 LED 显示

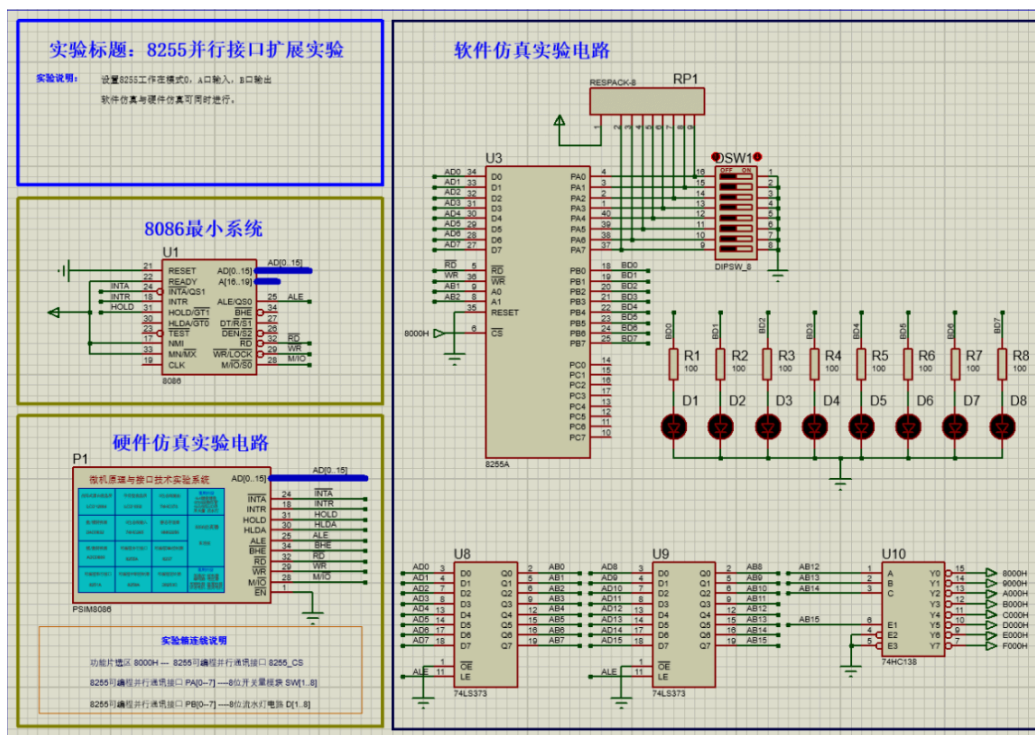


Figure 1: 电路原理图

我们通过配置 8255A 的工作方式，将其 PB 端口（IOB）和 PC 端口（IOC）分别用于控制 8 个 LED 灯中的不同部分。具体做法如下：

- 8255A 被设置为 A 口输入，B 口和 C 口输出（控制字 90H）。
- 程序每次从 A 口读取 8 位数据，将高 4 位（AL7 AL4）通过与操作（AND 0F0H）送到 PB 端口，控制前 4 个 LED（LED1 LED4）。
- 低 4 位（AL3 AL0）通过与操作（AND 0FH）送到 PC 端口，控制后 4 个 LED（LED5 LED8）。

这样，PB 端口的 4 位输出直接对应前 4 个 LED 的亮灭，PC 端口的 4 位输出对对应后 4 个 LED，实现了分端口独立控制。

```

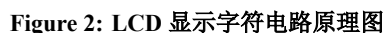
CODE    SEGMENT
ASSUME  CS:CODE

; 8255A 端口地址定义
IOCON   EQU 08006H      ; 控制口地址
IOA      EQU 08000H      ; A 口地址
IOB      EQU 08002H      ; B 口地址
IOC      EQU 08004H      ; C 口地址

START:
    MOV AL,90H           ; A 口输入，B/C 口输出
    MOV DX,IOCON
    OUT DX,AL

START1:
    MOV AL,0
  
```

1.2 LCD 显示字符



因此，只需在 DATA 中准备好学号与姓名对应的点阵并调用相应的字符串写入例程，LCD 即可在指定位置连续显示学号与姓名。

```

1  CODE      SEGMENT 'CODE'
2  ASSUME DS:DATA,CS:CODE,SS:STACK
3
4  LCD_CMD_WR EQU 9086H
5  LCD_DATA_WR1 EQU 9092H    ;写第一屏数据
6  LCD_DATA_WR2 EQU 9094H    ;写第二屏数据
7  LCD_DATA_WR0 EQU 9096H    ;写双屏数据
8  LCD_DATA_RD EQU 909EH
9  LCD_RST     EQU    9000H    ;复位
10 LCD_RST_OK   EQU    9080H    ;复位
11
12
13 LCD_ROW      EQU    0C0H    ;设置起始行
14 LCD_PAGE     EQU    0B8H    ;设置起始页
15 LCD_COLUMN   EQU    40H    ;设置起始列
16 LCD_PAGE_MAX EQU    08H    ;页数最大值
17 LCD_COLUMN_MAX EQU    40H    ;列数最大值
18
19 START:
20     MOV AX, DATA
21     MOV DS, AX
22
23     MOV AX, STACK
24     MOV SS, AX
25
26     MOV AX, TOP
27     MOV SP, AX
28
29     CALL LCD_INIT
30
31     CALL LCD_CLEAR
32
33 MAINLOOP:
34     CALL WRSTR_PROTEUS
35
36     CALL WRHZ
37
38     CALL WRSTR_PROTEUS2
39
40     CALL WRHZ2
41
42     JMP START
43
44 LCD_INIT:
45     MOV DX, LCD_RST
46     OUT DX, AL
47
48     MOV AL, 00H
49     MOV DX, LCD_RST_OK
50     OUT DX, AL
51
52     IN  AL, DX
53
54     MOV AL, LCD_ROW
55     CALL WRCMD
56
57     MOV AL, LCD_COLUMN
58     CALL WRCMD
59
60     MOV AL, LCD_PAGE
61     CALL WRCMD
62

```

```

63         MOV AL, 3FH
64         CALL WRCMD
65         RET
66
67 LCD_CLEAR:
68         MOV BL, 00H
69 LC_LOOP: MOV AL, BL
70         ADD AL, LCD_PAGE
71         CALL WRCMD
72         MOV AL, LCD_COLUMN
73         CALL WRCMD
74         MOV CX, LCD_COLUMN_MAX
75         MOV AL, 00H
76         MOV DX, LCD_DATA_WR0
77 LC_WDATA:
78         OUT DX, AL
79         LOOP LC_WDATA
80         INC BL
81         CMP BL, LCD_PAGE_MAX
82         JL LC_LOOP
83         RET
84
85 WRCMD:   PUSH DX
86         MOV DX, LCD_CMD_WR
87         OUT DX, AL
88         POP DX
89         RET
90
91 WRSTR_PROTEUS:
92         MOV AX, 8
93         MOV CX, 8
94         MOV BX, 0028H
95         LEA DI, ZF_1
96
97 WRSTR_LOOP:
98         CALL WRCHAR
99         ADD BX, AX
100        ADD DI, 16
101        LOOP WRSTR_LOOP
102        RET
103
104 WRSTR_PROTEUS2:
105        MOV AX, 8
106        MOV CX, 8
107        MOV BX, 0428H
108        LEA DI, ZF_9
109
110 WRSTR_LOOP2:
111        CALL WRCHAR
112        ADD BX, AX
113        ADD DI, 16
114        LOOP WRSTR_LOOP
115        RET
116
117 WRHZ:
118        MOV AX, 16
119        MOV CX, 3
120        MOV BX, 0230H
121        LEA DI, TABLE_HZ
122
123 WRHZ_LOOP:
124        CALL WRCHAR
125        ADD BX, AX
126        ADD DI, 32

```

```

125         LOOP WRHZ_LOOP
126     RET
127
128 WRHZ2:
129     MOV AX, 16
130     MOV CX, 3
131     MOV BX, 0630H
132     LEA DI, TABLE_HZ2
133
134 WRHZ_LOOP2:
135     CALL WRCHAR
136     ADD BX, AX
137     ADD DI, 32
138     LOOP WRHZ_LOOP2
139     RET
140
141
142 ;入口参数:
143 ;AX-->字符宽度, ASCII为8, 汉字为16
144 ;BL-->X地址, 0-127
145 ;BH-->Y地址, 0-7
146 ;DI-->字符首地址
147 WRCHAR: PUSH AX
148         PUSH BX
149         PUSH CX
150         PUSH DX
151         PUSH DI
152         MOV CX, AX
153         MOV AH, 02H
154         ADD BL, LCD_COLUMN
155         ADD BH, LCD_PAGE
156 WRCHAR_P:
157     MOV AL, BH
158     CALL WRCMD
159     TEST BL, 80H
160     JNZ WRCHAR_P0
161     MOV AL, BL
162     CALL WRCMD
163     MOV DX, LCD_DATA_WR1
164     JMP WRCHAR_P1
165 WRCHAR_P0:
166     MOV AL, BL
167     SUB AL, 40H
168     CALL WRCMD
169     MOV DX, LCD_DATA_WR2
170 WRCHAR_P1:
171     PUSH CX
172 WRBIT1: MOV AL, [DI]
173         OUT DX, AL
174         INC DI
175         LOOP WRBIT1
176         POP CX
177         INC BH
178         MOV AL, 00H
179         XCHG AL, AH
180         DEC AX
181         XCHG AL, AH
182         JNZ WRCHAR_P
183         POP DI
184         POP DX
185         POP CX
186         POP BX

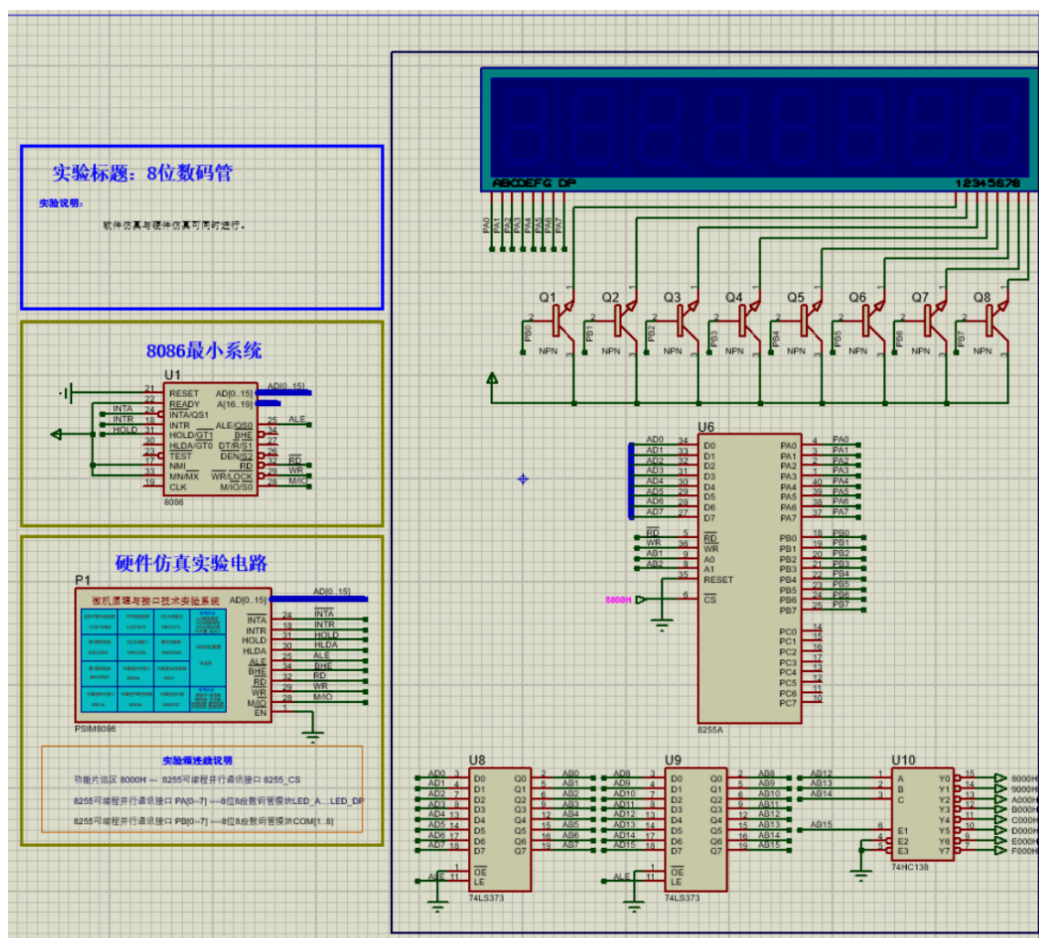
```

```

187         POP AX
188     WRRET:  RET
189
190     CODE    ENDS
191
192     STACK  SEGMENT 'STACK'
193     STA    DB 100 DUP(?)
194     TOP    EQU LENGTH STA
195     STACK  ENDS
196
197     END     START

```

1.3 8255A 控制 7 位数码管显示



我们通过将 8255A 配置为方式 0、A/B/C 口均为输出（控制字 80H），用端口 A 作段选（段码），端口 B 作位选（位选），实现对共阳/共阴（本实验为共阳）7 段数码管的多路复用扫描显示。程序在 DATA 段准备好每一位对应的段码表，循环 8 次依次显示 8 个数码管：先向端口 B 输出 0（消隐）关闭所有数码管，向端口 A 输出当前位的段码，再向端口 B 输出对应的位选信号选通当前数码管，短暂保持后右移位选掩码以选通下一位并读取下一个段码。通过高速循环刷新（轮询每位并不断重复），利用人眼的暂留效应得到完整的实验日期显示。为避免残影，单次切换时先消隐再写段码再点亮位选；位选由 BH 寄存器保存并每次右移以实现逐位扫描。

```

1  CODE      SEGMENT 'CODE'
2          ASSUME CS:CODE,SS:STACK,DS:DATA
3
4  IOCON     EQU 8006H      ; 8255控制寄存器端口
5  IOA       EQU 8000H      ; 8255端口A
6  IOB       EQU 8002H      ; 8255端口B
7  IOC       EQU 8004H      ; 8255端口C
8
9  START:
10         MOV AX, DATA    ; 初始化数据段
11         MOV DS, AX
12
13         MOV AX, STACK    ; 初始化堆栈段
14         MOV SS, AX
15
16         MOV AX, TOP      ; 设置堆栈指针 (这里有问题)
17         MOV SP, AX       ; 应该用 MOV SP, offset TOP
18
19
20  TEST_BU:
21         MOV AL,80H       ; 8255控制字: 方式0, A/B/C口都输出
22         MOV DX,IOCON
23         OUT DX,AL        ; 写入8255控制寄存器
24         NOP              ; 短暂延时
25
26  RESET:
27         LEA DI, TABLE   ; DI指向显示数据表
28         MOV CX,08H       ; 循环8次 (8个数码管)
29         MOV BH,80H       ; BH=10000000B (从最高位数码管开始)
30
31  DISPLA2:
32         MOV DX,IOB       ; 准备写端口B (位选)
33         MOV AL,00H       ; 先关闭所有数码管 (消隐)
34         OUT DX,AL        ; 输出到位选端口
35
36         MOV DX,IOA       ; 准备写端口A (段选)
37         MOV AL,[DI]      ; 读取显示数据
38         OUT DX,AL        ; 输出到段选端口
39
40         MOV DX,IOB       ; 再次写端口B
41         MOV AL,BH        ; 取位选信号
42         OUT DX,AL        ; 选通当前数码管
43
44         INC DI           ; 指向下一个显示数据
45         MOV AL,BH        ; 当前位选信号
46         SHR AL,1         ; 右移一位, 选通下一个数码管
47         MOV BH,AL        ; 保存到位选寄存器
48
49         LOOP DISPLA2     ; 循环显示8个数码管
50
51         JMP RESET        ; 无限循环
52
53  CODE      ENDS
54
55  STACK     SEGMENT 'STACK'
56  STA       DB 100 DUP(?) ; 100字节堆栈空间
57  TOP       EQU LENGTH STA ; TOP=100 (堆栈长度)
58  STACK     ENDS
59
60  DATA     SEGMENT 'DATA'
61  TABLE    DB 0F8H,0F9H,0A4H,0F9H,92H,0A4H,0C0H,0A4H
62

```

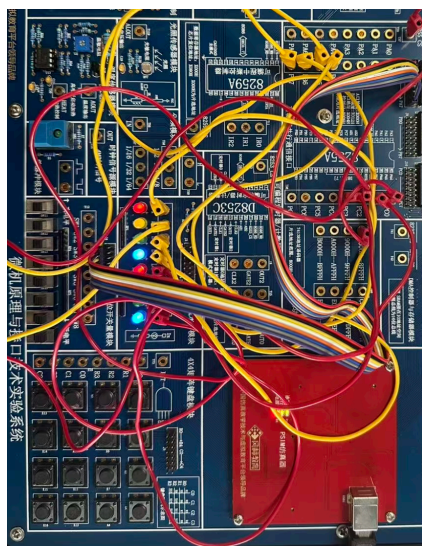
```

63 ; 数码管显示数据 (共阳极) :
64 DATA      ENDS
65 END START ; 程序结束, 从START开始执行

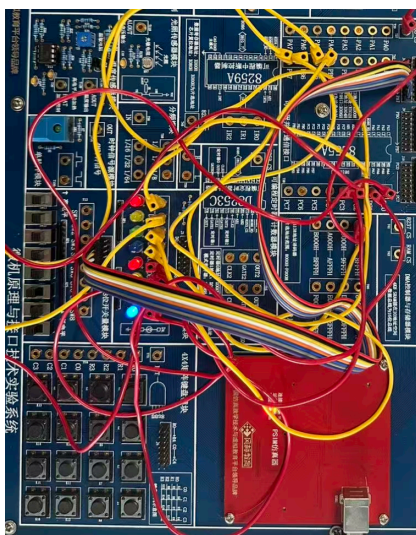
```

2 实验结果

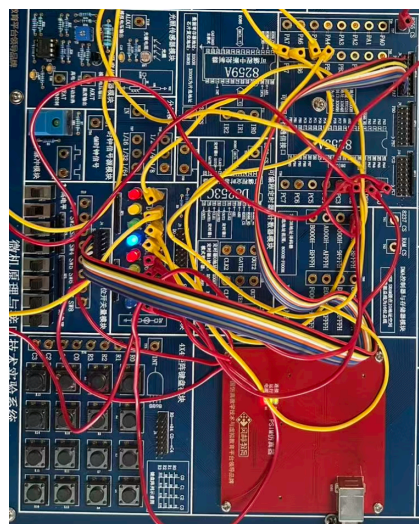
2.1 8255A 控制 LED 显示



(a) LED 显示效果 1



(b) LED 显示效果 2



(c) LED 显示效果 3

Figure 4: LED 显示效果

2.2 LCD 显示字符

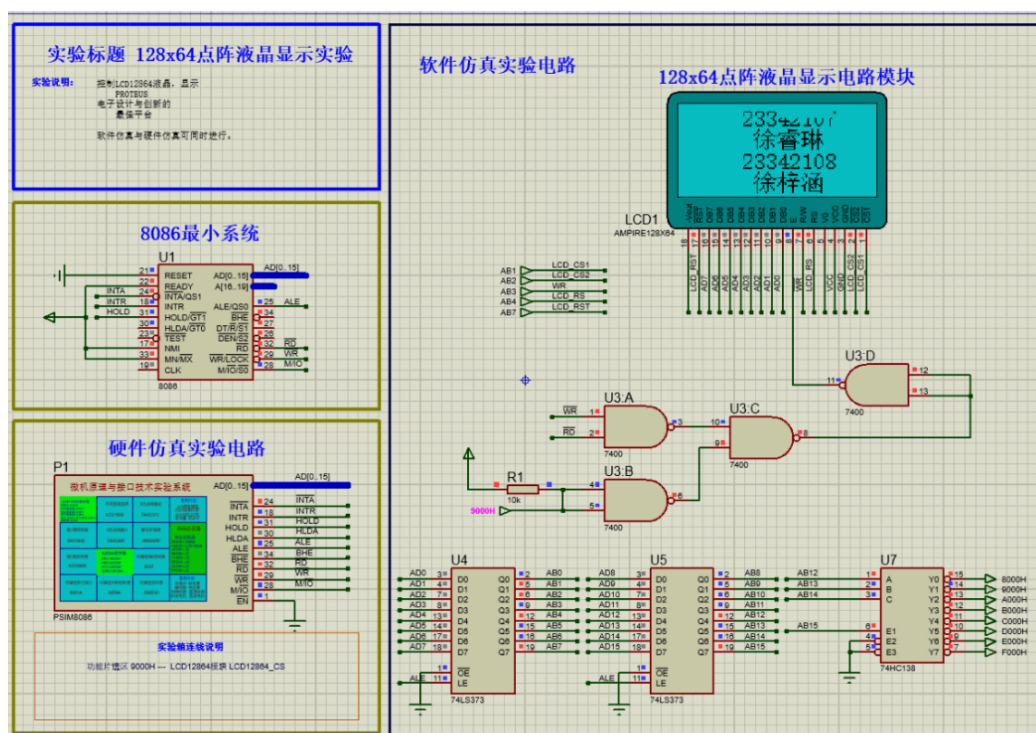


Figure 5: LCD 显示仿真软件截图

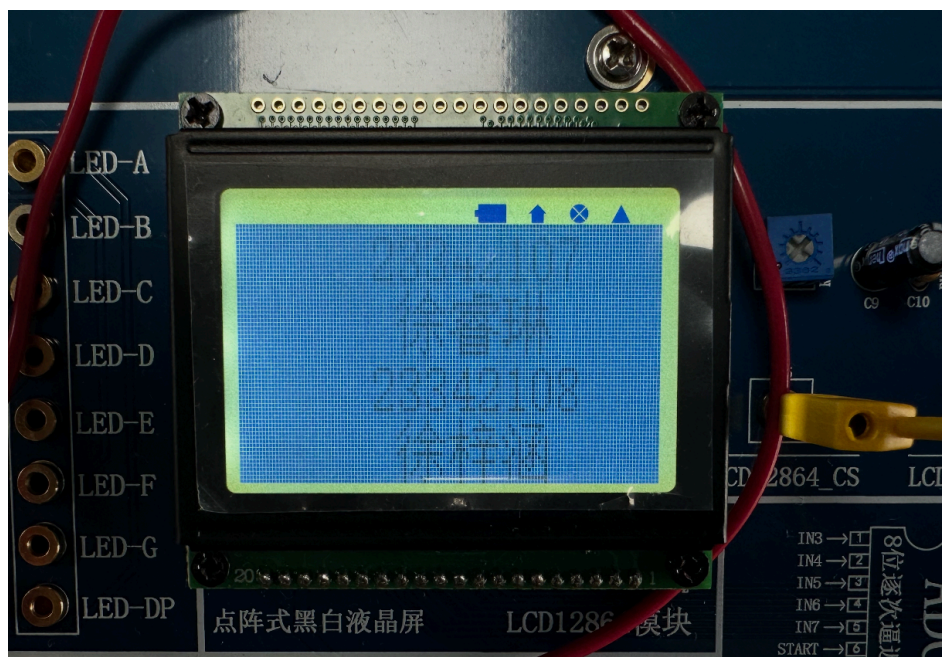


Figure 6: 学号-姓名 LCD 显示效果

2.3 8255A 控制 7 位数码管显示



Figure 7: 实验日期显示效果

3 思考与总结

3.1 实验总结

通过本次实验，我深入理解了 8255A 并行接口芯片的工作原理及其在 Mode 0 下的配置方法，并掌握了微机系统与外部显示设备交互的基本时序控制。

- **8255A 端口控制**: 在 LED 控制实验中，掌握了如何通过控制字将 8255A 的不同端口分别配置为输入和输出。理解了 CPU 通过数据总线读取开关状态并处理后输出到 LED 的完整流程。
- **LCD 液晶显示**: 熟悉了点阵式 LCD 的初始化序列、页/列地址设置以及汉字点阵的提取与写入方法。

特别是理解了“字模”是如何映射到屏幕像素点的，以及如何通过循环调用子程序实现字符串的连续显示。

- **数码管动态扫描：**在数码管实验中，深刻体会了“动态扫描”的设计思想。通过快速轮流选通各个数码管并送入对应的段码，利用人眼的视觉暂留效应实现了多位数字的同时显示。同时，实验中也验证了在位选切换前进行“消隐”操作对于消除显示残影（鬼影）的重要性。

3.2 鲁棒性提升方案：LCD 忙标志检测

3.2.1 问题分析

在当前的 LCD 显示程序中，我们在写入指令（WRCMD）或数据时，主要依赖于指令执行间的自然延时或简单的循环等待。然而，LCD 控制器（如 T6963C 或兼容芯片）处理指令内部需要一定的时间周期。如果 CPU 主频较高或写入速度过快，超过了 LCD 的处理能力，前一条指令尚未执行完毕就写入下一条，极易导致显示乱码、丢字或花屏。

为了提高程序的**鲁棒性**（Robustness），使其能适应不同速度的 CPU 或不同响应特性的 LCD 模块，标准的做法是在每次写入操作前，通过软件轮询 LCD 的状态寄存器，检测其是否处于“忙”状态。

3.2.2 改进代码实现

可以通过读取 LCD 的状态端口（通常与指令写入端口地址相同，但操作为读）来判断。对于 T6963C 控制器，状态字的低两位（STA0 和 STA1）通常表示指令读写和数据读写的能力。

改进思路：编写一个 CHECK_BUSY 子程序，在 WRCMD 和写数据前调用，确保 LCD 就绪后再进行输出。

```

1  ; -----
2  ; 改进后的 LCD 写指令子程序（增加忙检测）
3  ; -----
4  WRCMD_SAFE:
5      PUSH AX
6      PUSH DX
7
8      CALL CHECK_BUSY ; 关键改进：先检测忙状态，等待就绪
9
10     MOV DX, LCD_CMD_WR
11     OUT DX, AL      ; 确认不忙后，再安全写入指令
12
13     POP DX
14     POP AX
15     RET
16
17 ; -----
18 ; 判忙子程序 (Check Busy Status)
19 ; -----
20 CHECK_BUSY:
21     PUSH AX
22     PUSH DX
23     MOV DX, LCD_CMD_WR ; 读状态通常使用指令口的读操作
24
25 WAIT_FOR_IDLE:
26     IN AL, DX          ; 读取状态字
27     AND AL, 03H        ; 屏蔽高位，只保留 STA0, STA1
28                        ; STA0=1: 指令读写有效
29                        ; STA1=1: 数据读写有效
30     CMP AL, 03H        ; 检查是否两位都为 1
31     JNE WAIT_FOR_IDLE  ; 如果不等于 03H (即忙)，则循环等待
32
33     POP DX
34     POP AX

```

35

RET

通过加入上述 `CHECK_BUSY` 机制，程序不再依赖不可靠的硬延时，而是根据硬件的实际反馈来决定执行节奏。这不仅解决了潜在的时序冲突问题，也使得代码在移植到更高主频的实验箱或实际工程中时，依然能稳定运行。

附录 A 学号-姓名字符表

```

1 DATA SEGMENT 'DATA'
2 ;-- 文字: 2 --
3 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
4 ZF_1 DB 00H,70H,08H,08H,08H,08H,0F0H,00H,00H,30H,28H,24H,22H,21H,30H,00H
5
6
7 ;-- 文字: 3 --
8 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
9 ZF_2 DB 00H,30H,08H,08H,08H,88H,70H,00H,00H,18H,20H,21H,21H,22H,1CH,00H
10
11 ;-- 文字: 3 --
12 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
13 ZF_3 DB 00H,30H,08H,08H,08H,88H,70H,00H,00H,18H,20H,21H,21H,22H,1CH,00H
14
15 ;-- 文字: 4 --
16 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
17 ZF_4 DB 00H,00H,80H,40H,30H,0F8H,00H,00H,00H,06H,05H,24H,24H,3FH,24H,24H
18
19 ;-- 文字: 2 --
20 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
21 ZF_5 DB 00H,70H,08H,08H,08H,08H,0F0H,00H,00H,30H,28H,24H,22H,21H,30H,00H
22
23 ;-- 文字: 1 --
24 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
25 ZF_6 DB 00H,00H,10H,10H,0F8H,00H,00H,00H,00H,20H,20H,3FH,20H,20H,00H
26
27 ;-- 文字: 0 --
28 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
29 ZF_7 DB 00H,0E0H,10H,08H,08H,10H,0E0H,00H,00H,0FH,10H,20H,20H,10H,0FH,00H
30
31 ;-- 文字: 7 --
32 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
33 ZF_8 DB 00H,18H,08H,08H,88H,68H,18H,00H,00H,00H,3EH,01H,00H,00H,00H
34
35
36 ;-- 文字: 徐 --
37 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=16x16 --
38 TABLE_HZ DB 10H,88H,0C4H,33H,00H,20H,10H,28H,24H,0E3H,24H,28H,10H,20H,20H,00H
39 DB 01H,00H,0FFH,00H,20H,11H,0DH,41H,81H,7FH,01H,05H,09H,31H,00H,00H
40
41 ;-- 文字: 睿 --
42 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=16x16 --
43 DB 20H,18H,88H,0A8H,68H,28H,0A8H,6FH,0AAH,2AH,6AH,0AAH,0AH,28H,18H,00H
44 DB 04H,04H,02H,0FEH,0ABH,0ABH,0AAH,0AAH,0AAH,0ABH,0ABH,0FEH,02H,04H,04H,00H
45
46 ;-- 文字: 琳 --
47 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=16x16 --
48 DB 84H,84H,0FCH,84H,84H,10H,90H,0FFH,10H,00H,90H,0FFH,90H,10H,00H,00H
49 DB 10H,30H,1FH,08H,08H,06H,01H,0FFH,09H,06H,01H,0FFH,01H,06H,08H,00H
50
51 ;-- 文字: 2 --
52 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
53 ZF_9 DB 00H,70H,08H,08H,08H,08H,0F0H,00H,00H,30H,28H,24H,22H,21H,30H,00H
54
55 ;-- 文字: 3 --
56 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16 --
57 ZF_10 DB 00H,30H,08H,08H,08H,88H,70H,00H,00H,18H,20H,21H,21H,22H,1CH,00H
58
59 ;-- 文字: 3 --

```

```

60 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16  --
61 ZF_11 DB 00H,30H,08H,08H,08H,88H,70H,00H,00H,18H,20H,21H,22H,1CH,00H
62
63 ;-- 文字: 4  --
64 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16  --
65 ZF_12 DB 00H,00H,80H,40H,30H,0F8H,00H,00H,00H,06H,05H,24H,24H,3FH,24H,24H
66
67 ;-- 文字: 2  --
68 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16  --
69 ZF_13 DB 00H,70H,08H,08H,08H,08H,0F0H,00H,00H,30H,28H,24H,22H,21H,30H,00H
70
71 ;-- 文字: 1  --
72 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16  --
73 ZF_14 DB 00H,00H,10H,10H,0F8H,00H,00H,00H,00H,20H,20H,3FH,20H,20H,00H
74
75 ;-- 文字: 0  --
76 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16  --
77 ZF_15 DB 00H,0E0H,10H,08H,08H,10H,0E0H,00H,00H,0FH,10H,20H,20H,10H,0FH,00H
78
79 ;-- 文字: 8  --
80 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=8x16  --
81 ZF_16 DB 00H,70H,88H,08H,08H,88H,70H,00H,00H,1CH,22H,21H,21H,22H,1CH,00H
82
83
84 ;-- 文字: 徐  --
85 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=16x16  --
86 TABLE_HZ2 DB 10H,88H,0C4H,33H,00H,20H,10H,28H,24H,0E3H,24H,28H,10H,20H,20H,00H
87 DB 01H,00H,0FFH,00H,20H,11H,0DH,41H,81H,7FH,01H,05H,09H,31H,00H,00H
88
89 ;-- 文字: 梓  --
90 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=16x16  --
91 DB 10H,10H,0D0H,0FFH,90H,10H,40H,44H,54H,65H,0C6H,64H,54H,44H,40H,00H
92 DB 04H,03H,00H,0FFH,00H,03H,00H,04H,04H,04H,0FFH,04H,04H,04H,00H,00H
93
94 ;-- 文字: 涵  --
95 ;-- 宋体12; 此字体下对应的点阵为: 宽x高=16x16  --
96 DB 10H,60H,02H,8CH,00H,0F0H,02H,22H,42H,0F2H,4AH,26H,02H,0F0H,00H,00H
97 DB 04H,04H,7EH,01H,00H,7FH,44H,4AH,51H,4FH,41H,42H,44H,0FFH,00H,00H
98
99
100 DATA ENDS

```